

FarmPest Guide: A Dangerous Farm Insect Classification System

David Goanli
2702223582
Department of Computer Science
Bina Nusantara University
Jakarta, Indonesia

Kelson
2702245135
Department of Computer Science
Bina Nusantara University
Jakarta, Indonesia

Rafael Komala
2702227611
Department of Computer Science
Bina Nusantara University
Jakarta, Indonesia

Abstract—Dangerous agricultural insects pose a serious threat to crop productivity and quality, requiring fast and accurate identification methods to support effective pest management. This study aims to compare and evaluate three pretrained deep learning architectures, namely EfficientNet-B3, ResNet-50, and InceptionV3, in classifying dangerous farm insects using a computer vision approach. The dataset used consisted of 1591 insect images with 15 classes, which were processed through preprocessing and model training stages using transfer learning techniques. Performance evaluation was carried out based on classification reports, confusion matrices, and Grad-CAM visualizations to analyze accuracy, error patterns, and model interpretability. The results showed that EfficientNet-B3, which was fine-tuned on the top 30 layers, provided the best performance with an accuracy of 0.79 and more stable generalization capabilities compared to other models. The best model was then implemented in the FarmPest Guide application as a classification system for dangerous agricultural insects to support more efficient and sustainable agricultural practices.

Keywords—*deep learning, computer vision, inceptionv3, transfer learning, resnet50, efficientnet-b3, grad-cam, dangerous farm insects.*

I. INTRODUCTION

A. Background

Harmful insects pose a significant threat to agricultural productivity by damaging crops and reducing yield quality. Accurate identification of these insects is crucial for effective pest management and the selection of appropriate control methods. Traditionally, insect identification relies on expert knowledge and manual observation of morphological features, which can be time-consuming and impractical for large-scale agricultural environments. With the rapid development of computer vision and machine learning technologies, automated image-based insect identification has emerged as a promising solution to support modern and sustainable farming practices.

B. Problem Statement

The traditional process of insect identification still relies on manual observation by humans, which is time-consuming and difficult to apply on a large agricultural scale. Therefore, the main problem in this study is how to develop a computer vision-based system that is capable of classifying dangerous farm insects directly, accurately, and efficiently to support more effective pest management.

C. Objectives

This study aims to compare and evaluate three pretrained deep learning model architectures, namely EfficientNet-B3,

ResNet-50, and InceptionV3, in classifying dangerous farm insects by looking at the results of the classification report analysis, confusion matrix, and Grad-CAM interpretation to assess the performance. The best model results will be used for the FarmPest Guide application.

D. Significance

This research has practical significance by providing a deep learning-based classification system for dangerous farm insects that can help identify pests more quickly and accurately to support more effective agricultural management. Academically, this research provides comparative insights into the performance of different pretrained CNN architectures, which can serve as a reference for future research in the development of computer vision-based insect classification systems.

II. RELATED WORKS

Several studies have explored the application of deep learning and transfer learning techniques for insect pest classification. Vo *et al.* [1] investigated multiple optimization algorithms for EfficientNet-based transfer learning using a dataset of 1,591 agricultural insect images. The study showed that EfficientNet-B3 combined with the AdamW optimizer achieved the highest accuracy of 95.5%. However, CNN models trained from scratch performed poorly, and some misclassification issues still remained.

Syahputra *et al.* [2] compared several CNN architectures, including Xception, MobileNetV2, MobileNetV3, DenseNet-201, and InceptionV3, for insect pest classification using the IP-102 dataset. The experimental results indicated that DenseNet-201 with fine-tuning achieved the best performance with an accuracy of 70%. Nevertheless, the study reported limited exploration of hyperparameter tuning and deployment efficiency.

In the context of household insect recognition, Alhadidi *et al.* [3] proposed a hybrid system combining YOLOv8 for detection and deep transfer learning models for classification. ResNet50 and VGG19 were used as feature extractors, followed by SVM for classification on a dataset of 6,548 images. The results demonstrated that ResNet50 outperformed VGG19 with an accuracy of 95.11%. However, the dataset covered only a limited number of insect species and required manual labeling.

Kar *et al.* [4] explored self-supervised learning (SSL) approaches to improve insect pest classification, particularly under limited labeled data conditions. Using SSL methods such as NNCLR, BYOL, and Barlow Twins with ResNet

architectures, the study reported improved accuracy and better generalization compared to supervised learning, especially on datasets such as IP-102. Despite its advantages, SSL required higher computational resources and longer training time.

Finally, Cao *et al.* [5] focused on recognizing common field insects using transfer learning with pretrained CNN models such as VGG16, VGG19, and Inception. The dataset consisted of nine insect classes, and data augmentation techniques, including DCGAN, were applied. The highest accuracy of 97.39% was achieved using VGG19. However, the study was conducted on a small dataset under controlled conditions, limiting its generalizability.

III. METHODOLOGY

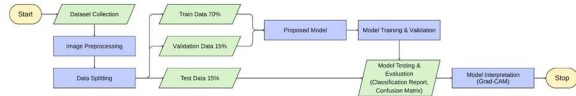


Figure 1. Methodology flowchart.

A. Dataset

The dataset used in this study is the Dangerous Farm Insects dataset obtained from Kaggle and uploaded by Tarun R Jain in 2023 under a CC0 Public Domain license. The dataset consists of 1591 labeled images representing 15 classes of agricultural insect pests that commonly threaten crop production. The images exhibit variations in background, lighting, and insect appearance, making the classification task more challenging. The purpose of this dataset is to facilitate research in pest identification, crop protection, and the development of automated computer vision systems for early detection of harmful insects in agriculture.

B. Preprocessing

The preprocessing steps performed include removing broken images, handling duplicate images both within the same class and across different classes, adjusting brightness levels, image sharpening, and resizing all images to 300×300 pixels. After that, each image is also processed using the `preprocess_input()` function according to the model used, namely EfficientNet-B3, ResNet-50, and InceptionV3.

C. Pretrained Model Selection

In this project, we compared 3 pretrained models that performed well in previous research on dangerous farm insect classification, namely:

1) EfficientNet-B3

EfficientNet-B3 is a CNN model that optimizes accuracy and computational efficiency by using a compound scaling approach on depth, width, and resolution. After experimentation, EfficientNet-B3 achieved a train accuracy of 0.8446 and a train loss of 0.5089, with a val accuracy of 0.7569 and a val loss of 0.7817 at epoch 40.

2) ResNet-50

ResNet-50 is a 50-layer deep CNN model that uses residual connections to overcome the vanishing gradient problem in training very deep networks. After experimentation, ResNet-50 achieved a train accuracy of 0.9466 and a train loss of 0.2273, with a val

accuracy of 0.7294 and a val loss of 0.8894 at epoch 31.

3) InceptionV3

InceptionV3 is a CNN architecture that uses Inception modules (a combination of parallel 1×1, 3×3, and 5×5 convolutions) to capture features at various scales efficiently, then reduces the number of parameters through techniques such as factorized convolutions and dimensionality reduction. After experimentation, InceptionV3 achieved a train accuracy of 0.9276 and a train loss of 0.2983, with a val accuracy of 0.6789 and a val loss of 0.9813 at epoch 37.

Based on these results, we selected the EfficientNet-B3 pretrained model for our application because it is more stable and achieves the best validation performance, with a validation accuracy of 0.7569 and a validation loss of 0.7817.

D. Model Architecture

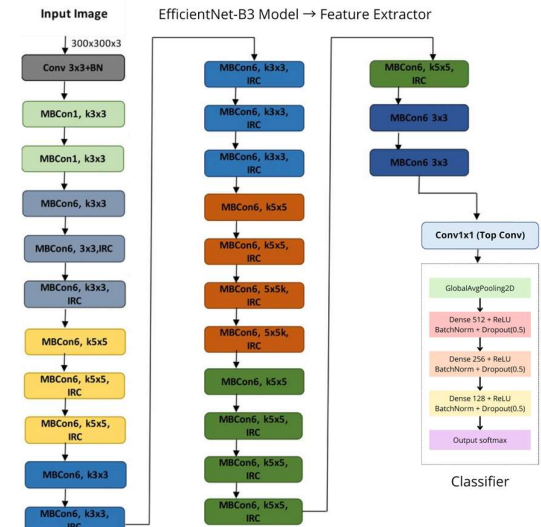


Figure 2. Proposed EfficientNet-B3 model architecture with custom classifier.

1) Pretrained EfficientNet-B3 Model (Feature Extractor):

The EfficientNet-B3 architecture was developed using the compound scaling method with a coefficient of $\phi = 3$ to increase the depth, width, and resolution of the architecture in a balanced manner. This method can be expressed in the following equation:

$$\text{depth: } d = \alpha^\phi, \text{ width: } w = \beta^\phi, \text{ resolution: } r = \gamma^\phi \\ \text{s.t. } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \alpha \geq 1, \beta \geq 1, \gamma \geq 1 \dots (1)$$

When the input image resolution is enlarged, the network requires increased depth to expand the receptive field and increased channel width to effectively capture more detailed visual patterns. Based on the original EfficientNet paper, the compound scaling coefficient values used are $\alpha = 1.2$, $\beta = 1.1$, and $\gamma = 1.15$, which form the basis for the development of all EfficientNet variants, starting from the EfficientNet-B0 baseline model.

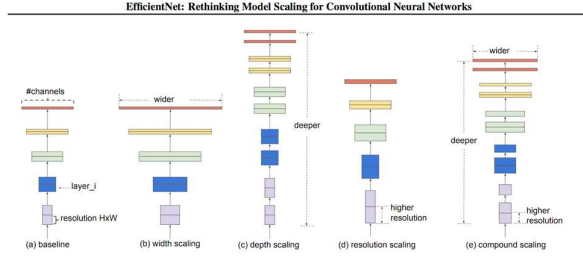


Figure 3. Compound scaling method from EfficientNet-B0.

- i) Depth Scaling (d)
The depth parameter is used to set the number of layers or the number of MBConv block iterations in the neural network.
- ii) Width Scaling (w)
The width parameter is used to control the number of channels in each convolution layer. The larger the width value, the more feature maps can be learned in each layer.
- iii) Resolution Scaling (r)
The resolution parameter sets the spatial size of the image input that will enter the neural network. Higher resolutions allow the model to capture finer visual details, but also increase the size of the feature map, which impacts computation and memory consumption.

Based on the compound scaling formula, the theoretical scaling values obtained for EfficientNet-B3 are $\alpha^3 \approx 1.73$ for depth, $\beta^3 \approx 1.33$ for width, and $\gamma^3 \approx 1.52$ for resolution, which indicate the magnitude of improvement made by EfficientNet-B3 from the EfficientNet-B0 baseline model. However, in the actual implementation of the EfficientNet-B3 architecture, the scaling results are not used raw, but are rounded first because the number of channels, number of layers, and input resolution in the CNN architecture parameters can only be discrete integers and are aligned with computational efficiency. This adjustment is necessary to ensure that the increase in model complexity remains controlled, so that the floating point operations (FLOPs) of EfficientNet-B3 are in the range of approximately 1.8 billion FLOPs. This means that the model's efficiency increases without excessive growth in computational costs.

Stage	Operator	Resolution	#Channel	#Layers
1	Conv 3×3	300 × 300	40	1
2	MBConv1, k=3×3	150 × 150	24	2
3	MBConv6, k=3×3	150 × 150	32	3
4	MBConv6, k=5×5	75 × 75	48	3
5	MBConv6, k=3×3	38 × 38	96	4

6	MBConv6, k=5×5	38 × 38	136	4
7	MBConv6, k=5×5	19 × 19	232	5
8	MBConv6, k=3×3	10 × 10	384	2
9	Conv 1×1 (Top Conv)	10 × 10	1536	1

Table 1. Details of the EfficientNet-B3 architecture.

Based on the EfficientNet-B3 architecture structure shown in Figure 2 and Table 1, the following is an explanation of the image processing flow and the functions of each layer in the model:

- i) Initial Convolution Layer (Conv 3×3)
The input image shape measuring 300×300×3 will be processed by a convolution layer with a 3×3 kernel and a stride of 2 to prepare the initial representation of features such as edges, texture, color, and so on before entering the MBConv core block. The resulting output shape is 150×150×40.
- ii) Mobile Inverted Bottleneck Convolution (MBConv)
This section is the core block of EfficientNet-B3 that will be used repeatedly in stages 2 to 8. MBConv is a development of the inverted residual block found in MobileNetV2, with the addition of a Squeeze-and-Excitation (SE) mechanism to improve feature representation quality. It consists of 4 processing stages, namely:
 - 1) Expansion (1×1 Convolution)
The input channel will be expanded using 1×1 convolution with an expansion ratio corresponding to the MBConv name. For example, it will be expanded 6× for MBConv6 stage 3, from 150×150×24 to 150×150×144 feature map size. The goal is to increase feature representation capacity.
 - 2) Depthwise Convolution (k × k)
Convolution with a k×k kernel is performed separately on each channel and read by sliding the kernel, making the computation much more efficient because the number of parameters is only k×k×Channel. For example, in stage 3, a 3×3 kernel is shifted over a 150×150×144 feature map, where each channel has its own 3×3 kernel. This aims to capture local spatial patterns in each channel.
 - 3) Squeeze-and-Excitation (SE) Block
This mechanism is responsible for providing channel-wise attention to the most informative channels. In the squeeze stage, Global Average Pooling is used to summarize the feature map. For example, in the feature map that has been enlarged in the expansion stage, namely 150×150×144, it will be squeezed into 1×1×144. In the excitation stage, the squeezed vector will go through 2 fully connected layers with nonlinear and sigmoid activation functions in the final layer, which will produce a 1×1×144 vector in the form of weight values for the importance

level of each channel. Finally, in the scale stage, the original feature map ($150 \times 150 \times 144$) will be multiplied by the weights from the excitation stage (weights sized $1 \times 1 \times 144$) to clarify which channels are strong and which are weak. The output from this stage is a weighted feature map with a size of $150 \times 150 \times 144$.

4) Projection (1×1 Convolution)

The feature map will be compressed back to the specified output shape using 1×1 convolution. For example, the feature map resulting from $150 \times 150 \times 144$ scaling will be projected to $150 \times 150 \times 32$. The goal is to reduce the channel dimension and retain the feature information that has been enriched by the previous process.

iii) Skip Connection

This skip connection mechanism will be applied to MBConv blocks that meet two conditions, namely, stride = 1 and the number of input channels = the number of output channels. The formula can be expressed in an equation, namely:

$$Output = X + MBConv(X) \dots (2)$$

Since the spatial dimensions and number of feature map channels remain consistent, the input feature map (X) can be added directly to the MBConv block output. This ensures that feature information from the previous layer is preserved and can be effectively passed on to the next layer.

The final result of stage 8 on the pretrained EfficientNet-B3 model is a $10 \times 10 \times 384$ feature map that represents high-level features extracted from the entire convolution process. Next, it will be processed by the top convolution layer in the form of a 1×1 convolution that serves to increase the number of channels from 384 to 1536. This layer is followed by Batch Normalization and the Swish activation function to stabilize and enrich the feature representation. Thus, the final output of the EfficientNet-B3 backbone used as a feature extractor is a feature map measuring $10 \times 10 \times 1536$.

In this study, the weight of the pretrained EfficientNet-B3 model will be used as a feature extractor. Most of its layers will be frozen, and the top 30 layers will be unfrozen for the fine-tuning process to adjust the feature representation results to the characteristics of the dataset used, namely, dangerous farm insect classification.

2) Custom Classifier

Layer	Input Shape	Output Shape
GlobalAveragePooling2D	$10 \times 10 \times 1536$	1536
Dense (512) + ReLU	1536	512
Dropout	512	512
Dense (256) + ReLU	512	256

Dropout	256	256
Dense (128) + ReLU	256	128
Dropout	128	128
Output Dense (Softmax)	128	15 class

Table 2. Details of the custom classifier architecture.

i) GlobalAveragePooling2D

Convert the 3D feature map to 1D, take the average of each channel, and retain the most important information.

ii) Dense 512 + ReLU + BatchNorm + Dropout 0.5

Taking abstract features and then converting them into a more compact representation. BatchNorm for stabilizing the activation distribution. Dropout to prevent overfitting.

iii) Dense 256 + ReLU + BatchNorm + Dropout 0.5

Re-process features from the previous layer and clarify the separation between classes.

iv) Dense 128 + ReLU + BatchNorm + Dropout 0.5

Refining features to make them easy to map to output.

v) Output Dense (Softmax)

Converting the representation into probabilities for each insect class and softmax activation ensures that the total probability is 1.

The EfficientNet-B3 model used in this study has a total of 10,783,535 parameters, with 3,554,170 trainable parameters and 7,229,365 non-trainable parameters. After adding a custom classifier, the total number of parameters increased to 11,740,222, with 4,509,065 trainable parameters and 7,231,157 non-trainable parameters.

E. Training Setup

The dataset will be divided into 70% training data, 15% validation data, and 15% test data. All images will be resized to 300×300 pixels with a batch size of 32. Before training, each batch is preprocessed using the appropriate preprocess_input() function based on the model being trained so that the image format and normalization comply with ImageNet standards. The training dataset will be shuffled by 1000 samples so that the model does not learn data sequence patterns, and the entire dataset will use prefetch (AUTOTUNE) to speed up the pipeline and reduce bottlenecks during training.

The model will use the AdamW optimizer with a learning rate of $1e-4$ and weight decay of $5e-6$, as well as the sparse_categorical_crossentropy loss function. Then, class weight is used to handle class imbalance so that the model is not biased. In addition, the EarlyStopping and ReduceLROnPlateau callbacks are also used to prevent model stagnation. The model was trained for 40 epochs.

F. Evaluation Metrics

The model was evaluated using a classification report (accuracy, precision, recall, and F1-score) to see the performance of each class, a confusion matrix to observe the

number of correct and incorrect predictions per class, and Grad-CAM to see the parts of the image that the model focused on when making predictions.

IV. IMPLEMENTATION & RESULTS

A. System Details

The experiment was conducted using Visual Studio Code software. The hardware used was a system with a 12th Gen Intel Core i7-12650H (2.30 GHz) processor, 16 GB RAM (4800 MT/s), and a GPU with 4 GB VRAM capacity.

B. Experiments

Metric	EfficientNet-B3	ResNet-50	InceptionV3
Train Accuracy	0.8446	0.9466	0.9276
Validation Accuracy	0.7569	0.7294	0.6789
Test Accuracy	0.7941	0.7605	0.7059
Train Loss	0.5089	0.2273	0.2983
Validation Loss	0.7817	0.8894	0.9813
Test Loss	0.6576	0.8277	0.968

Table 3. Performance comparison of EfficientNet-B3, ResNet-50, and InceptionV3 on the dangerous farm insect classification dataset.

C. Visualisation

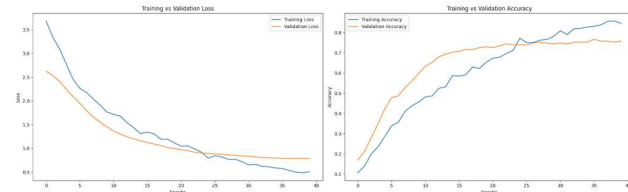


Figure 4. Training vs validation loss and training vs validation accuracy of the EfficientNet-B3 model.

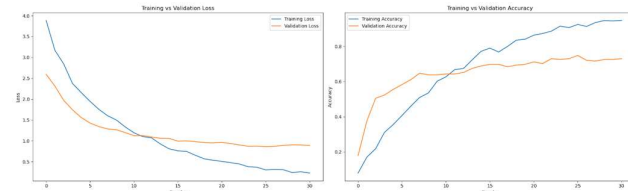


Figure 5. Training vs validation loss and training vs validation accuracy of the ResNet-50 model.

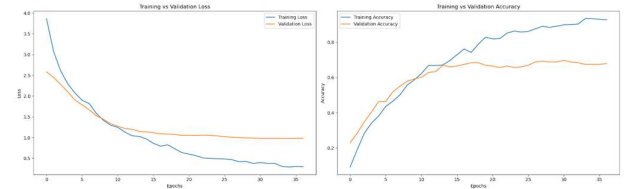


Figure 6. Training vs validation loss and training vs validation accuracy of the InceptionV3 model.

D. Result Obtained

Classification Report:	precision	recall	f1-score	support
Africanized Honey Bees (Killer Bees)	1.00	0.93	0.97	15
Aphids	0.77	0.77	0.77	13
Armyworms	0.38	0.45	0.42	11
Brown Marmorated Stink Bugs	1.00	0.83	0.91	18
Cabbage Loopers	1.00	0.76	0.87	17
Citrus Canker	0.84	1.00	0.91	16
Colorado Potato Beetles	0.94	0.94	0.94	17
Corn Borers	0.60	0.67	0.63	18
Corn Earworms	0.73	0.65	0.69	17
Fall Armyworms	0.26	0.36	0.30	14
Fruit Flies	0.93	0.93	0.93	15
Spider Mites	0.56	0.89	0.71	18
Thrips	0.78	0.82	0.80	17
Tomato Hornworms	1.00	1.00	1.00	17
Western Corn Rootworms	0.85	0.73	0.79	17
accuracy			0.79	238
macro avg	0.80	0.78	0.79	238
weighted avg	0.82	0.79	0.80	238

Figure 7. Classification report for EfficientNet-B3.

Classification Report:	precision	recall	f1-score	support
Africanized Honey Bees (Killer Bees)	1.00	0.87	0.93	15
Aphids	0.83	0.77	0.80	13
Armyworms	0.62	0.73	0.67	11
Brown Marmorated Stink Bugs	0.88	0.78	0.82	18
Cabbage Loopers	0.90	0.53	0.67	17
Citrus Canker	1.00	0.88	0.93	16
Colorado Potato Beetles	1.00	0.96	0.97	17
Corn Borers	0.57	0.56	0.56	18
Corn Earworms	0.52	0.65	0.58	17
Fall Armyworms	0.42	0.57	0.48	14
Fruit Flies	0.93	0.93	0.93	15
Spider Mites	0.88	0.83	0.86	18
Thrips	0.75	0.71	0.73	17
Tomato Hornworms	0.68	0.88	0.77	17
Western Corn Rootworms	0.80	0.80	0.80	17
accuracy			0.76	238
macro avg	0.78	0.76	0.77	238
weighted avg	0.79	0.76	0.77	238

Figure 8. Classification report for ResNet-50.

Classification Report:	precision	recall	f1-score	support
Africanized Honey Bees (Killer Bees)	0.93	0.93	0.93	15
Aphids	0.67	0.62	0.64	13
Armyworms	0.38	0.27	0.30	11
Brown Marmorated Stink Bugs	0.93	0.72	0.81	18
Cabbage Loopers	0.89	0.47	0.62	17
Citrus Canker	0.84	1.00	0.91	16
Colorado Potato Beetles	0.83	0.88	0.86	17
Corn Borers	0.65	0.61	0.63	18
Corn Earworms	0.53	0.47	0.50	17
Fall Armyworms	0.29	0.36	0.32	14
Fruit Flies	0.93	0.93	0.93	15
Spider Mites	0.76	0.72	0.74	18
Thrips	0.71	0.69	0.69	17
Tomato Hornworms	0.70	0.94	0.80	17
Western Corn Rootworms	0.61	0.93	0.74	15
accuracy			0.71	238
macro avg	0.71	0.70	0.69	238
weighted avg	0.72	0.71	0.70	238

Figure 9. Classification report for InceptionV3.

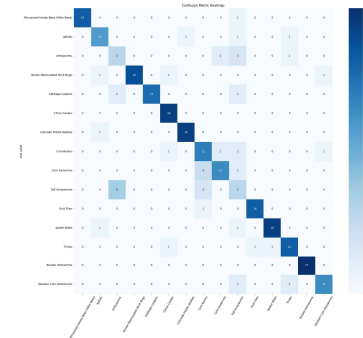


Figure 10. Confusion matrix for EfficientNet-B3.

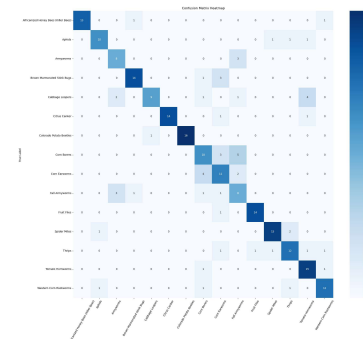


Figure 11. Confusion matrix for ResNet-50.

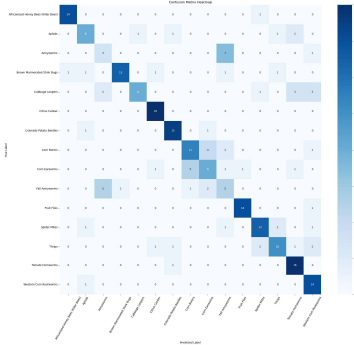


Figure 12. Confusion matrix for InceptionV3.

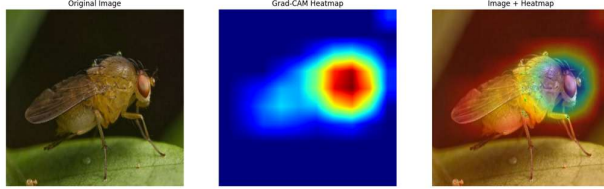


Figure 13. Example of Grad-CAM visualization for the Fruit Flies class using EfficientNet-B3.

V. DISCUSSION & LIMITATIONS

A. Analysis of Performance

Based on Figures 4, 5, and 6, the results on the train, validation, and test sets show that the EfficientNet-B3 model with a custom classifier performs the best due to its good generalization ability, stability, and lack of significant overfitting. In comparison, the ResNet-50 and InceptionV3 models show more overfitting, albeit slightly, due to the larger gap between training and validation and testing.

From the classification report results in Figures 7, 8, and 9, the EfficientNet-B3 model has the best performance with an accuracy of 0.79, followed by ResNet-50 with an accuracy of 0.76, and finally InceptionV3 with an accuracy of 0.71. From the confusion matrix in Figures 10, 11, and 12, it can be seen that the three models experience the same error pattern, especially in Corn Borers, Corn Earworms, and Fall Armyworms, which are interchanged because they have similar body shapes and image backgrounds. In addition, Armyworms are also often confused with Fall Armyworms because they are still in the same genus, so their patterns and body textures are similar. These four classes produced low F1-scores across all models.

Overall, the difference in performance between the three models was influenced by the suitability of the architecture to the size and characteristics of the dataset. EfficientNet-B3 excelled because its compound scaling design was able to capture visual details more efficiently, as seen from its stable F1-score in almost all classes. Although ResNet-50 has skip connections that aid training stability, its capacity is less than optimal for complex insect shape variations, resulting in performance below EfficientNet-B3. Meanwhile, InceptionV3 showed the lowest performance because its more complex architecture was not fully absorbed by the small dataset size.

B. Challenges

- The small size of the dataset, consisting of only 1,591 images, limits the visual variation of each class, making it difficult for the model to learn rich features. This has an impact on classes with similar morphology or those from closely related genera, increasing the risk of prediction errors.
- Computational limitations due to the entire training process being carried out using personal devices without dedicated GPUs slowed down training time, especially for EfficientNet-B3, which took 1 hour, 9 minutes, and 46 seconds, limiting our ability to explore hyperparameter combinations or try more complex architectures.

C. Trade-Offs

- Final model selection**
EfficientNet-B3 was selected as the main model despite its longer training time, as it provided the most stable performance and highest accuracy. The trade-off was that prediction quality was more important than computational efficiency because the model would be used in real-world applications.
- Handling dataset imbalance**
Actually, the dataset is not too imbalanced, so class weighting is used to handle this because it is faster and does not add computational load like data augmentation. However, class weighting has a trade-off, namely, suboptimal performance in classes with similar morphology, so that test accuracy does not increase significantly.

VI. CONCLUSION & FUTURE WORK

This study compares 3 fine-tuned CNN architectures, namely EfficientNet-B3, ResNet-50, and InceptionV3, for classifying dangerous farm insects. Based on the experimental results, the best model is EfficientNet-B3 with an accuracy of 0.79. However, the prediction performance is still lacking in four classes, namely Armyworms (F1-score 0.42), Fall Armyworms (F1-score 0.30), Corn Borers (F1-score 0.63), and Corn Earworms (F1-score 0.69). These four classes have morphological similarities and are in closely related genera, so the model often misclassifies them in real cases. Nevertheless, these results are still better than ResNet-50 and InceptionV3, because EfficientNet-B3 with its compound scaling is able to capture visual details more efficiently, so that the F1-score remains stable in most other classes. For future work, experiments with data augmentation should be considered to increase visual variation so that the model can learn more robust representations and reduce errors due to morphological similarities between classes.

REFERENCES

- S. N. Alhadidi, "Detecting and classifying household insects in Iraq by using transfer learning models," *Journal of Advanced Research in Applied Sciences and Engineering Technology*, vol. 50, no. 1, pp. 21–33, 2024. <https://doi.org/10.37934/araset.50.1.2133>.
- X. Cao, Z. Wei, Y. Gao, and Y. Huo, "Recognition of common insect in field based on deep learning," *Journal of Physics: Conference Series*, vol. 1634, no. 1, Art. no.

012034, 2020. <https://doi.org/10.1088/1742-6596/1634/1/012034>.

- [3] S. Kar et al., “Self-supervised learning improves classification of agriculturally important insect pests in plants,” *Plant Phenome Journal*, vol. 6, no. 1, Art. no. e20079, 2023. <https://doi.org/10.1002/ppj2.20079>.
- [4] A. P. Syahputra, A. C. Siregar, and R. W. S. Insani, “Comparison of CNN models with transfer learning in the classification of insect pests,” *Indonesian Journal of Computing and Cybernetics Systems*, vol. 17, no. 1, pp. 103–114, 2023. <https://doi.org/10.22146/ijccs.80956>.
- [5] Tarundalal, “Dangerous insects dataset,” Kaggle, [Online]. <https://www.kaggle.com/datasets/tarundalal/dangerous-insects-dataset>.
- [6] H.-T. Vo, N. T. Nguyen, K. M. Chau, H. L. Lam, and P. T. Pham, “Exploring multiple optimization algorithms in transfer learning with EfficientNet models for agricultural insect classification,” *CTU Journal of Innovation and Sustainable Development*, vol. 16, Special Issue: ISDS, pp. 35–41, 2024. <https://doi.org/10.22144/ctujoisd.2024.320>.
- [7] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, PMLR, pp. 6105–6114, 2019. <https://doi.org/10.48550/arXiv.1905.11946>.

VII. APPENDIX

A. Team Contribution Statement

Team Member	David Goanli	Kelson	Rafael Komala
EDA		✓	
Preprocessing			✓
Modelling	✓	✓	✓
Evaluation & Result	✓	✓	✓
Apps Deployment	✓		
Final Report		✓	✓
Demo Video	✓		
Presentation Slides		✓	✓

Table 4. Team contribution statements.

B. Supporting Documents

No.	Document	Link
1.	GitHub Repo	https://github.com/Luceatt/Final-Project-Deep-Learning
2.	Application	https://huggingface.co/spaces/DG226/DeepLearning
3.	Demo Video	OneDrive Demo Video
4.	Presentation Slides	Canva Presentation Slides

Table 5. Project supporting documents and links.

C. Screenshots and Code Snippets

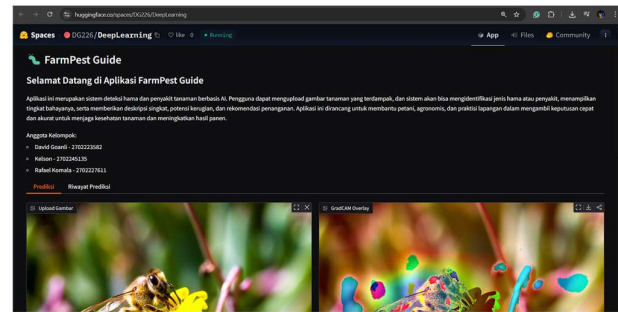


Figure 14. User Interface of the FarmPest Guide Application.

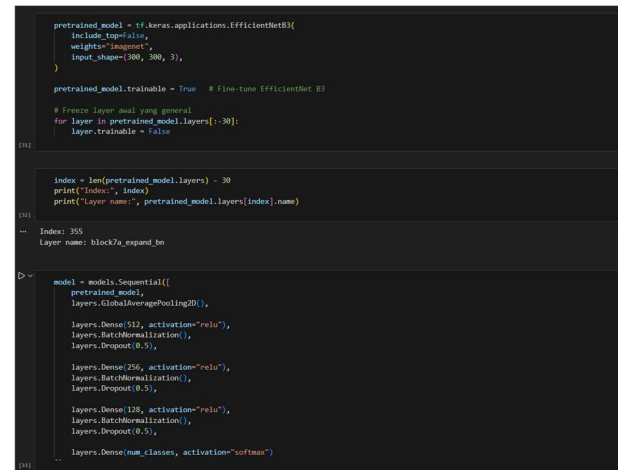


Figure 15. Code Snippet Example.